BSc. Software Development
Year II

Project Report-CA3

Daniel Belov, Tadhg Brennan
C00306133, C00308963
9/12/2025

Contents

Project Introduction

In this project, Tadhg Brennan and I (Daniel Belov), work together in order to implement working algorithms using multiple data structures, including hashing and graphs. This project was created in order to build and display our understanding of data structure skills including how they operate and how they can be applied to real world problems in software development.

Part 1: Hash Table Application

Replace this with your Hash Table application submission under the following headings:

(a) What is a Hash Table ?

A hash table is a data structure to store elements and provides quick and efficient access to them

(b) Description of your Hash Table application

Linear probing: elements are inputted onto a table based on the remainder of the element mod a specific number. If space is taken and element has no room for input (collision), element will move over to the next available slot (probing). The number of probes is how many times the slot is used.

(c) Table of the Data used under the following headings:

User Name	ASCII Integer	Index after Hash function applied
sam	321	1
katy	441	1
bill	419	19
james	528	8
pete	430	10
joe	318	18
amy	327	7
kim	321	1
jess	437	17
tom	336	16

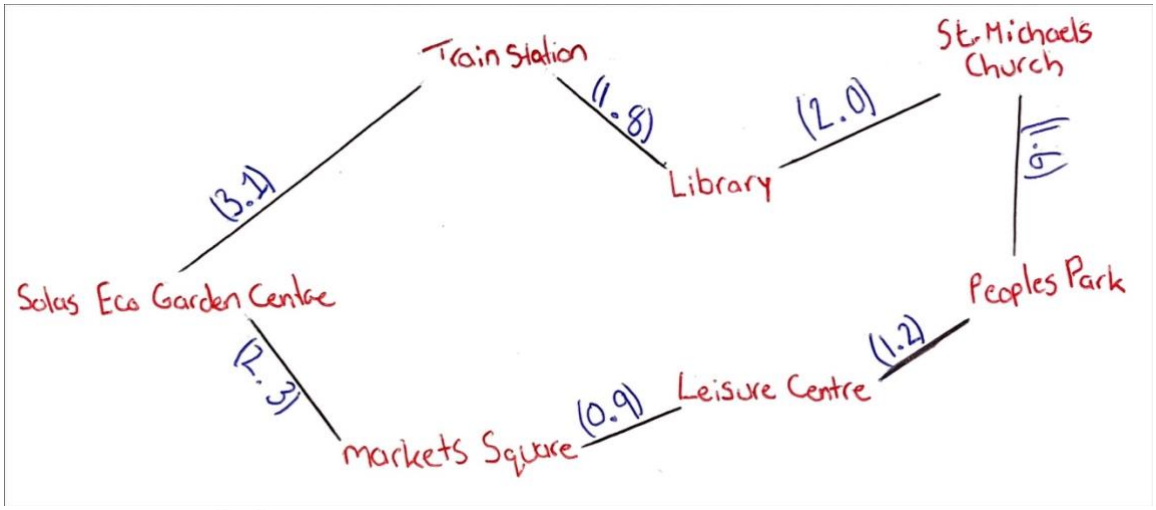
(d) Diagram of the Hash Table produced- with collisions highlighted.

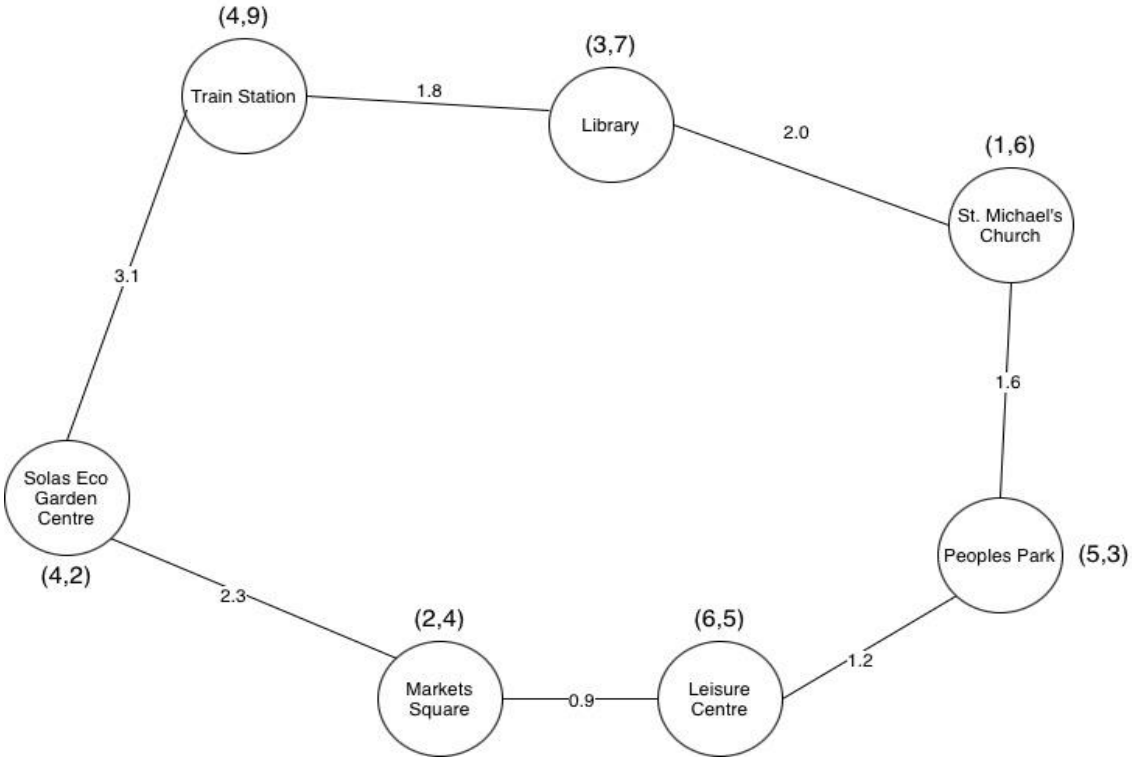
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
	1	1	1				7	8		10						16	17	18	19

Part 2: Graph Application

Replace this with your Graph application submission under the following headings:

(a) Diagram of Map of Tourist Sites:



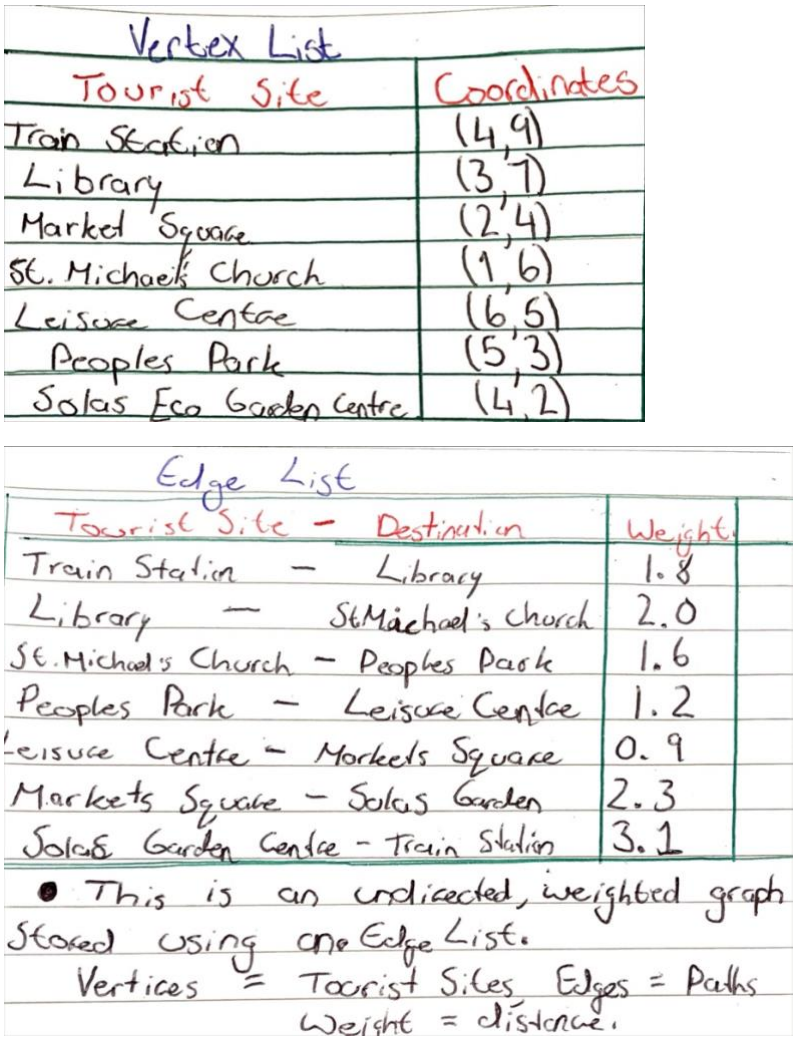


(b) Description of Data Structure used to store your Map and any helper variables

The Data Structure used to store this map is an "Edge List". An Edge List is an unordered list of all edges in the graph. Each two vertices (tourist sites) are stored along with an edge (walking path) between them. The weight of each edge Is stored as the walking distance between two tourist sites. This approach is efficient as it only stores actual real-world pathways, rather than every possible connection between

sites. Helper variables that will be implemented are, Vertex List which will store each tourist sites and their appropriate coordinates, Edge List which stores the paths between tourist t sites and the distance between them, vertexCount will keep track of the total number of tourist sites and edgeCount will track the amount of walking paths.

(c) Diagram of your Map stored in your chosen Data Structure
This section MUST be handwritten (unless you have a handwriting waiver)



Vertex List

Tourist Site	Coordinates
Train Station	(4,9)
Library	(3,7)
Market Square	(2,4)
St. Michael's Church	(1,6)
Leisure Centre	(6,5)
Peoples Park	(5,3)
Solas Eco Garden Centre	(4,2)

Edge List

Tourist Site – Destination	Weight
Train Station – Library	1.8
Library – StMichael's church	2.0
St. Michael's Church – Peoples Park	1.6
Peoples Park – Leisure Centre	1.2
Leisure Centre – Markets Square	0.9
Markets Square – Solas Garden	2.3
Solas Garden Centre – Train Station	3.1

- This is an undirected, weighted graph stored using one Edge List.

Vertices = Tourist Sites Edges = Paths
Weight = distance.

(d) Minimum Spanning Tree of your Graph:
Indicate which algorithm you used to calculate the MST and show the order of the chosen edges.

Algorithm Used: Kruskal's Minimum Spanning Tree Algorithm

Edge List in ascending order:

Weight	Tourist Site
0.9	Leisure Centre – Markets Square
1.2	Peoples Park – Leisure Centre
1.6	St. Michaels Church – Peoples Park
1.8	Train Station – Library
2.0	Library – St. Michaels Church
2.3	Markets Square – Solas Eco Garden Centre
3.1	Solas Eco Garden Centre – Train Station (*REJECTED EDGE*)

Step by step, Applying Kruskal's Algorithm:

Step	Weight	Edge	Cycle?	Add/Reject
1.	0.9	Leisure Centre – Markets Square	No	ADD
2.	1.2	Peoples Park – Leisure Centre	No	ADD
3.	1.6	St. Michaels Church – Peoples Park	No	ADD
4.	1.8	Train Station - Library	No	ADD
5.	2.0	Library – St. Michael's Church	No	ADD
6.	2.3	Markets Square – Solas Eco Garden Centre	No	ADD
7.	3.1	Solas Eco Garden Centre – Train Station	Yes	REJECT

Chosen Edges:

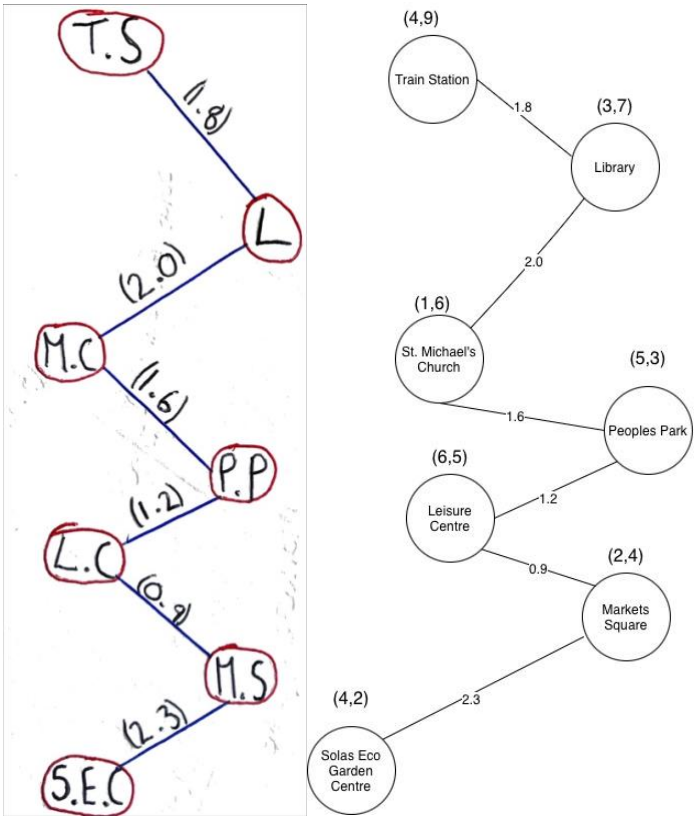
1. Leisure Centre – Market Square (0.9)
2. Peoples Park – Leisure Centre (1.2)
3. St. Michael's Church – Peoples Park (1.6)
4. Train Station – Library (1.8)
5. Library – St. Michael's Church (2.0)
6. Markets Square – Solas Eco Garden Centre (2.3)

Rejected Edge(s):

7. Solas Eco Garden Centre – Train Station (3.1), rejected because if included it would create a full cycle

Total MST edges = 6, this is correct considering we have 7 vertices (n-1).

Spanning Tree:



Keys:

"T.S" = Train Station
"L" = Library
"M.S" = St. Michael's Church
"P.P" = Peoples Park
"L.C" = Leisure Centre
"M.C" = Markets Square
"S.E.C" = Solas Eco Garden Centre

(e) Pseudocode of Algorithms

Search():

```
Search(site):
  FOR each V in VertexList(site):
    IF V.name = site THEN
```

```
        OUTPUT V.name
        OUTPUT V.coordinates
        RETURN
      ENDIF
  ENDFOR
  OUTPUT "Site not found"
END
```

Insert():

```
Insert(site1, site2, weight):
  CREATE newEDGE
  newEdge.start = site1
  newEdge.end = site2
  newEdge.weight = weight
  ADD newEdge to EdgeList
END
```

AllCons():

```
AllCons(site):
  FOR each E in EdgeList
    IF E.start = site THEN
      OUTPUT E.end
    ELSE IF E.end = site THEN
      OUTPUT E.start
    ENDIF
  ENDFOR
END
```

Closest():

```
Closest(site):
  minWeight = INFINITY
  closestSite = " "

  FOR each E IN EdgeList
    IF E.start = site THEN
      IF E.weight < minWeight THEN
        minWeight = E.weight
        closestSite = E.end
```

```
        ENDIF
      ELSE IF E.end = site THEN
        IF E.weight < minWeight THEN
          minWeight = E.weight
          closestSite = E.start
        ENDIF
      ENDIF
    ENDFOR
    OUTPUT closestSite
END
```

Part 3: Code

- A copy of fully documented code
- a list of all the functions/routines which have been used in the code(one line),
- Sample execution screenshots of outputs generated for each operation using the Map/Graph data you provided

Copy of Code:

Main (Driver Class):

```
//TIP To <b>Run</b> code, press <shortcut actionId="Run"/> or
// click the <icon src="AllIcons.Actions.Execute"/> icon in the gutter.
public class Main
{
    public static void main(String[] args)
    {
        //create a new graph
        Graph g = new Graph();

        //add vertices (tourist sites) to the graph
        g.addVertex("Train Station", 4, 9);
        g.addVertex("Library", 3, 7);
        g.addVertex("Market Square", 2, 4);
        g.addVertex("St. Michael's Church", 1, 6);
        g.addVertex("Leisure Centre", 6, 5);
        g.addVertex("Peoples Park", 5, 3);
        g.addVertex("Solas Eco Garden Centre", 4, 2);

        //insert edges (walking paths) between the sites
        g.Insert("Leisure Centre", "Market Square", 0.9);
        g.Insert("Peoples Park", "Leisure Centre", 1.2);
        g.Insert("St. Michael's Church", "Peoples Park", 1.6);
        g.Insert("Train Station", "Library", 1.8);
        g.Insert("Library", "St. Michael's Church", 2.0);
        g.Insert("Market Square", "Solas Eco Garden Centre", 2.3);

        //test for the implemented algorithms with sample data
        System.out.println();
        g.Search("Library");

        System.out.println();
        g.AllCons("Leisure Centre");

        System.out.println();
        g.Closest("Peoples Park");
    }
}
```

Vertex Class:

```
public class Vertex
{
    //Attributes for Vertex class
```

```
    String name;
    public int x;
    public int y;

    //Constructor for Vertex class
    public Vertex(String name, int x, int y)
    {
        this.name = name;
        this.x = x;
        this.y = y;
    }
}
```

Edge Class:

```
public class Edge
{
    //Attributes for Edge class
    String start;
    String end;
    double weight;

    //Constructor for Edge class
    Edge (String start, String end, double weight)
    {
        this.start = start;
        this.end = end;
        this.weight = weight;
    }
}
```

Graph Class:

```
public class Graph
{
    //Attributes for Graph class
    Vertex[] VertexList = new Vertex[20];    // max 20 vertices (tourist
sites)
    Edge[] EdgeList = new Edge[40];        // max 40 edges (walking paths
between sites)

    //Counter attributes for Graph class
    int vertexCount = 0;
    int edgeCount = 0;

    //Method to add a vertex to the graph
    public void addVertex(String name, int x, int y)
    {
        VertexList[vertexCount] = new Vertex(name, x, y);
        vertexCount++;
    }

    //Algorithm 1 - Search(site)
    // Algorithm to output details of a given site
    void Search(String site)
    {
        System.out.println("Search() output:");
```

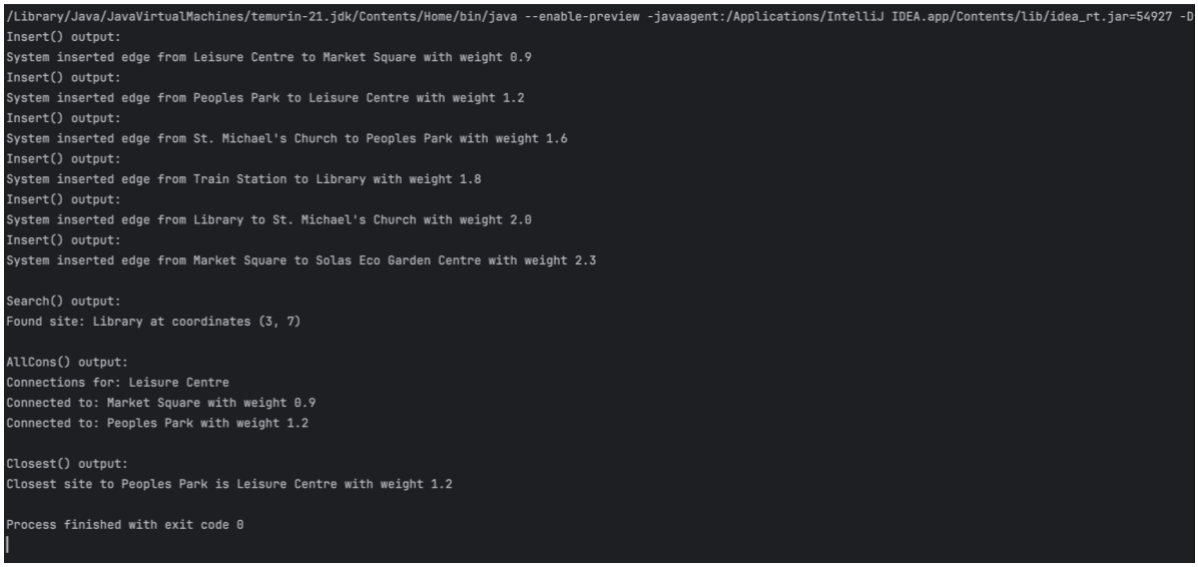
```
        // Loop through all vertices to find the site
        for(int i = 0; i < vertexCount; i++)
        {
            // Check if the current vertex matches the site
            if(VertexList[i].name.equals(site))
            {
                System.out.println("Found site: " + site + " at coordinates
(" + VertexList[i].x + ", " + VertexList[i].y + ")");
                return;
            }
        }
        // prints ONLY if site is not found
        System.out.println("Site not found in the graph.");
    }


    //Algorithm 2 - Insert(String site1, String site2, double weight)
    // Algorithm to insert a new edge between two sites with a given weight
    void Insert(String site1, String site2, double weight)
    {
        System.out.println("Insert() output:");

        // Create a new edge and add it to the EdgeList
        EdgeList[edgeCount] = new Edge(site1, site2, weight);
        edgeCount++;
        System.out.println("System inserted edge from " + site1 + " to " +
site2 + " with weight " + weight);
    }

    //Algorithm 3 - AllCons(String site)
    // Algorithm to output names of all sites connected to the given site
    void AllCons(String site)
    {
        System.out.println("AllCons() output:");

        // Loops through all the edges to find the connection for any given
site
        System.out.println("Connections for: " + site);
        for(int i = 0; i < edgeCount; i++)
        {
            // Check if the current edge is connected to the site
            if (EdgeList[i].start.equals(site))
            {
                System.out.println("Connected to: " + EdgeList[i].end + "
with weight " + EdgeList[i].weight);
            }
            else if (EdgeList[i].end.equals(site))
            {
                System.out.println("Connected to: " + EdgeList[i].start + "
with weight " + EdgeList[i].weight);
            }
        }
    }

    //Algorithm 4 - Closest(site)
    // Algorithm to find and output the closest connected site to the given
site
    void Closest(String site)
    {
        System.out.println("Closest() output:");
```

```
        // Variables to track the minimum weight and closest site
        double minWeight = Double.MAX_VALUE;
        String closestSite = null;

        // Loops through all edges to find the closest connected site
        for(int i = 0; i < edgeCount; i++)
        {
            // IF the current edge is connected to the site AND its weight
is less than the current minimum weight
            if (EdgeList[i].start.equals(site) && EdgeList[i].weight <
minWeight)
            {
                minWeight = EdgeList[i].weight;
                closestSite = EdgeList[i].end;
            }

            else if (EdgeList[i].end.equals(site) && EdgeList[i].weight <
minWeight)
            {
                minWeight = EdgeList[i].weight;
                closestSite = EdgeList[i].start;
            }
        }
        // Print the closest site and its weight
        if (closestSite != null)
        {
            System.out.println("Closest site to " + site + " is " +
closestSite + " with weight " + minWeight);
        }
        // prints if no connections found
        else
        {
            System.out.println("No connections found for site: " + site);
        }
    }
}
```

List of all the functions/routines which have been used in the code:

- **Algorithm 1** - Search(site):
  - Algorithm to output details of a given site
- **Algorithm 2** - Insert(String site1, String site2, double weight):
  - Algorithm to insert a new edge between two sites with a given weight
- **Algorithm 3** - AllCons(String site) :
  - Algorithm to output names of all sites connected to the given site
- **Algorithm 4** - Closest(site):
  - Algorithm to find and output the closest connected site to the given

Output:


/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/java --enable-preview -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=54927 -D
Insert() output:
System inserted edge from Leisure Centre to Market Square with weight 0.9
Insert() output:
System inserted edge from Peoples Park to Leisure Centre with weight 1.2
Insert() output:
System inserted edge from St. Michael's Church to Peoples Park with weight 1.6
Insert() output:
System inserted edge from Train Station to Library with weight 1.8
Insert() output:
System inserted edge from Library to St. Michael's Church with weight 2.0
Insert() output:
System inserted edge from Market Square to Solas Eco Garden Centre with weight 2.3
Search() output:
Found site: Library at coordinates (3, 7)
AllCons() output:
Connections for: Leisure Centre
Connected to: Market Square with weight 0.9
Connected to: Peoples Park with weight 1.2
Closest() output:
Closest site to Peoples Park is Leisure Centre with weight 1.2
Process finished with exit code 0

References

[1] Aine Byrne (2025). Data Structures and Graphs Lecture Slides. SETU.

[2] OpenAI ChatGPT (2025). Text-based assistance, Tadhg Brennan, 06 December 2025. Topic: MST tracing and Java implementation.

[3] “Kruskal MST Visualzation.” *Usfca.edu*, 2025, www.cs.usfca.edu/~galles/visualization/Kruskal.html. Accessed 7 Dec. 2025.

[4] “Minimum Spanning Tree (Prim’s, Kruskal’s) - VisuAlgo.” Visualgo.net, 2021, visualgo.net/en/mst?utm_. Accessed 7 Dec. 2025.